KORIAMC

KoriaQuest

Documentation technique · Quêtes hebdomadaires et journalières

Version 1.0.0 · Spigot 1.8.8 · Java 8
Mise à jour le 25 septembre 2026

Sommaire

1. 1. Compilation
2. 2. Principe
3. 3. Architecture
4. 4. Cycle de vie
5. 5. Configuration
6. 6. Modèle de données
7. 7. Stockage
8. 8. Objectifs
9. 9. Anti-abus
10. 10. Commandes et permissions
11. 11. Menus
12. 12. API
13. 13. PlaceholderAPI
14. 14. Points d'attention

Quêtes hebdomadaires de faction et de joueur pour le serveur **KoriaMC**.

Plateforme	Spigot / PaperSpigot **1.8.8**
Java	8
Package racine	`fr.koriamc.quest`
Classe principale	`fr.koriamc.quest.KoriaQuest`
Dépendance obligatoire	**KoriaFactions** (`depend`)
Dépendances optionnelles	Vault (récompenses `ARGENT`), PlaceholderAPI

1. Compilation

KoriaFactions est une dépendance `system` : le pom pointe sur `../KoriaFactions/target/KoriaFactions-1.0.0.jar` . **Compiler KoriaFactions d'abord**, puis :

```
mvn clean package
```

Le jar est produit dans `target/KoriaQuest-1.0.0.jar` . Les deux jars doivent être installés ensemble : KoriaQuest utilise des méthodes ajoutées à KoriaFactions pour lui (points, `getTerritoryType` , `SPAWNER_METADATA`).

2. Principe

- **Deux périodes indépendantes**, dans le fuseau de `cycle.fuseau` :
  - **semaine des quêtes de faction** : commence au jour et à l'heure de `config.json` → `cycle` ;
  - **période des quêtes joueur** : chaque jour à `cycle-joueur.heure` (`periode` = `QUOTIDIEN` , par défaut), ou chaque semaine avec les factions (`HEBDOMADAIRE`).

  Chaque période a son identifiant ; toute donnée d'une autre période est remise à zéro au premier accès.
- **Quêtes de faction** : un seul tirage par semaine, **commun à toutes les factions** (`quetes-faction.nombre` quêtes par difficulté). Chaque faction a sa propre progression, alimentée par tous ses membres. L'objectif vaut `objectif + par-membre × taille de la faction` , calculé au premier accès de la semaine.
- **Quêtes joueur** : chaque jour, le joueur choisit une difficulté ; `quetes-joueur.nombre` quêtes sont tirées pour lui seul.
- **Récompenses** : par quête terminée et par palier (nombre de quêtes terminées dans la période). `POINTS:n` crédite les points de faction de KoriaFactions.
- **Accès depuis KoriaFactions** : `/f quests` (commande de KoriaFactions) exécute `/quetes faction` .
- **Changement de faction** : la progression de faction reste à la faction ; les contributions d'un joueur qui part restent donc à l'ancienne. Ses quêtes joueur le suivent et leurs points vont à sa faction du moment.

3. Architecture

Package	Rôle
`(racine)`	`KoriaQuest` : cycle de vie, configuration, accès aux services, dépôt Vault
`api`	`KoriaQuestAPI` (façade statique)
`api.event`	`QuestProgressEvent`, `QuestCompleteEvent`, `BonusEarnedEvent`, `WeeklyResetEvent`, `PlayerQuestsResetEvent`
`command`, `command.impl`	commande dynamique `/quetes`, sous-commandes joueur et admin
`config`	`JsonFile`, `JsonWriter`, `ConfigDocs`, `Messages` (repris de KoriaFactions), `Settings`, `QuestCatalog`, `RewardSettings`
`model`	`QuestDefinition`, `QuestProgress`, `PlayerData`, `FactionData`, `CycleState`, `QuestScope`
`objective`	`ObjectiveType`, `ObjectiveRegistry`, `ProgressContext`
`progress`	`QuestManager`, `CycleManager`, `ProgressService`, `QuestView`, `AntiAbuse`
`reward`	`RewardService`
`listener`	`ObjectiveListener` (évènements Bukkit → actions de quête), `SessionListener`
`gui`	`Menu`, `QuestListMenu`, `PlayerQuestsMenu`, `DifficultyMenu`, `FactionQuestsMenu`, éditeur, `MenuManager`, `ChatPrompt`
`hook`	`FactionHook` (seule classe qui interroge KoriaFactions), `VaultHook`, `PapiHook`, `QuestExpansion`
`storage`	`DataManager`, `JsonStorage`, `SqlStorage`, `DataSnapshot`, `Storage`
`util`	`Text`, `Items`, `MaterialSet`, `Sounds`, `Nms` (repris de KoriaFactions)

Services

Classe	Responsabilité
`QuestManager`	données à jour de la période (`player(...)` : période joueur, `faction(...)` : semaine), éligibilité d'une faction, choix de difficulté avec confirmation, tirages pondérés, rappel des quêtes non terminées, réinitialisations admin
`CycleManager`	calcul du début de la semaine (factions) et de la période joueur (jour ou semaine), vérification toutes les 30 s, renouvellements (`newWeek`, `newPlayerPeriod`), temps restants
`ProgressService`	réception des actions, filtres (mondes, zones, cibles), avancement, notifications, livraison, réussite, récompenses et paliers
`QuestView`	variables d'affichage d'une quête (menus, messages, PlaceholderAPI)
`RewardService`	exécution des récompenses, description lisible, récompenses en attente des absents
`AntiAbuse`	blocs posés, mobs de spawner / d'œuf, kills répétés, AFK, et état des objectifs `ELEVER`, `BRASSER`, `PARCOURIR`
`FactionHook`	faction d'un joueur (en ligne ou non), membres, points, zone d'une position, relation avec une victime

4. Cycle de vie

Démarrage

1. Création des 6 `JsonFile` et des services.
2. Chargement de la configuration : réglages, messages, récompenses, puis catalogue des quêtes. Les problèmes (quête invalide, difficulté inconnue...) sont écrits dans la console.
3. `descriptions.json` → `LISEZMOI.txt`.
4. Chargement des données. **En cas d'échec, le plugin se désactive.**
5. Relecture des blocs posés (`blocs-poses.dat`), PlaceholderAPI, écouteurs, commande.
6. `CycleManager.start()` : au tout premier démarrage, ou si le début calculé de la semaine (factions) ou de la période joueur a changé pendant l'arrêt, la période concernée est renouvelée (sans annonce au démarrage).
7. Sauvegarde automatique et tâche du temps de jeu (chaque minute).

Arrêt

Désenregistrement de la commande, arrêt des tâches, fermeture des menus, écriture de `blocs-poses.dat`, sauvegarde synchrone.

Rechargement (`/quetes recharger` , éditeur)

Relecture des fichiers, des commandes et de la sauvegarde automatique, puis `cycles.check()`. **Modifier le jour, l'heure ou le fuseau de `cycle` démarre aussitôt une nouvelle semaine ; modifier `cycle-joueur` (ou le fuseau) renouvelle aussitôt les quêtes joueur.** Les quêtes déjà tirées ne sont pas retirées : une quête supprimée du fichier disparaît simplement de l'affichage et ne progresse plus.

5. Configuration

Même moteur que KoriaFactions (`JsonFile`) : défauts du jar pour les options absentes, sections libres, messages d'erreur avec ligne et colonne, éditeur `/quetes config` , explications dans `descriptions.json` et `LISEZMOI.txt`.

Fichier	Contenu	Sections libres
`config.json`	stockage, commande et sous-commandes, `cycle` , `cycle-joueur` , `difficultes` , `quetes-faction` , `quetes-joueur` , `anti-abus` , `notifications` , `affichage` , `titres`	`difficultes` , `quetes-faction.nombre`
`quests_faction.json`	`{ difficulté: { id: quête } }`	toutes (`*`)
`quests_player.json`	`{ difficulté: { id: quête } }`	toutes (`*`)
`rewards.json`	`faction` (`par-quete` , `paliers`), `joueur.<difficulté>` (`par-quete` , `paliers`)	`faction.par-quete` , `faction.paliers` , `joueur` , `joueur.*.paliers`
`messages.json`	textes (clés en anglais)	
`menus.json`	`joueur` , `difficulte` , `faction` , `editeur`	`*.objets` , `editeur.fichiers`

Les quêtes et les paliers sont des **objets indexés par identifiant** et non des listes : l'éditeur peut ainsi les ouvrir, les copier (`AJOUTER-ENTREE`) et les supprimer.

Difficultés

Les identifiants de `config.json` → `difficultes` (ordre conservé) font foi. Une section de difficulté inconnue dans un fichier de quêtes est ignorée avec un avertissement. Ajouter une difficulté demande : son entrée dans `difficultes` , ses quêtes, ses récompenses, `quetes-faction.nombre.<id>` et un objet `CHOISIR:<id>` dans le menu `difficulte` .

Schéma d'une quête

Clé	Type	Rôle
`actif`	booléen	`false` retire la quête du tirage
`nom`	texte	`{quete}`
`type`	texte	identifiant d'un `ObjectiveType`
`cibles`	liste	selon le type : matériaux, noms d'entités ou relations ; vide = tout
`objectif`	entier > 0	quantité à atteindre
`par-membre`	entier	faction uniquement : ajouté par membre
`mondes`	liste	mondes acceptés (vide = tous)
`zones`	liste	`WILDERNESS` , `SAFEZONE` , `WARZONE` , `OWN` , `ALLY` , `ENEMY` (vide = toutes)
`poids`	entier ≥ 1	poids dans le tirage
`icone`	matériau	`{icone}`
`description`	liste	`{description}` ; `{objectif}` y est remplacé
`recompenses`	liste d'actions	remplace les récompenses de la difficulté (vide = celles de `rewards.json`)

`QuestCatalog` valide chaque quête : type connu, objectif positif, matériaux reconnus (une quête dont **toutes** les cibles sont invalides est ignorée pour ne pas tout accepter), zones connues. Clé interne d'une quête : `difficulte.id` (`QuestDefinition.key()`), c'est elle qui est enregistrée dans la progression.

Récompenses (`rewards.json`)

Un bloc contient `recompenses` (actions), `texte` (affichage, vide = généré) et, côté faction, `cible` :

Cible	Destinataires des récompenses personnelles
`CONTRIBUTEURS`	membres actuels ayant contribué à la quête (au palier : à une quête de la semaine)
`MEMBRES_EN_LIGNE`	membres connectés
`MEMBRES`	tous les membres, les absents les reçoivent à la connexion
`AUCUNE`	personne (seuls `POINTS` et `ANNONCE` s'appliquent)

Les paliers sont indexés par un nombre de quêtes terminées (`"3"`). Tous les paliers atteints et pas encore reçus sont donnés ; le bonus affiché dans les menus est le palier égal au nombre de quêtes, sinon le plus haut.

6. Modèle de données

`QuestProgress`

`quest` (clé `difficulte.id`), `target`, `progress` (borné entre 0 et `target`), `completedAt`, `contributions` (faction : UUID → quantité apportée).

`PlayerData`

`uuid`, `name`, `cycle` (identifiant de la période joueur), `difficulty` (null = pas choisie), `quests`, `tiers` (paliers reçus), `pending` (actions de récompense déjà résolues, à exécuter à la prochaine connexion). `reset(cycle)` vide la période mais garde `pending`.

`FactionData`

`faction` (identifiant KoriaFactions, stable), `cycle`, `quests`, `tiers`. Supprimée quand la faction est dissoute (vérification au tick suivant de `FactionDisbandEvent`, pour gérer les dissolutions forcées).

`CycleState`

Factions : `cycle` (numéro de semaine), `weekStart` (ms), `factionQuests` (clés tirées pour toutes les factions). Joueurs : `playerStart` (début de la période en ms) et `playerCycle`, identifiant de période égal à `max(playerStart, précédent + 1)` : toujours croissant, même pour un renouvellement forcé dans la même période.

7. Stockage

Identique à KoriaFactions (`stockage.type` : `JSON`, `SQLITE`, `MYSQL` ; `MYSQL` sert aussi pour MariaDB, pilote `com.mysql.jdbc.Driver` fourni par Spigot). Tout est en mémoire, sérialisé par Gson sur le thread principal puis écrit par le thread `KoriaQuest-Save`.

Donnée	JSON (`data/`)	SQL (préfixe `kq_` par défaut)
Joueurs	`players.json`	`kq_players`
Factions	`factions.json`	`kq_factions`
Cycle	`cycle.json` (clé `cycle`)	`kq_meta`

Chaque table : `(id PRIMARY KEY, data)` avec le JSON de l'entité ; seules les lignes modifiées sont écrites.

Purge automatique : ne sont sauvegardés que les joueurs ayant des données de la période en cours ou des récompenses en attente, et les factions ayant des quêtes de la semaine en cours. Les anciennes périodes disparaissent donc du stockage à la sauvegarde suivante.

`blocs-poses.dat` (dossier du plugin) : liste binaire des blocs posés par les joueurs (noms des mondes puis positions compactées sur un `long`).

8. Objectifs

Types intégrés (`ObjectiveRegistry`)

Type	Cibles	Source	Quantité
`CASSER_BLOC`	matériaux	`BlockBreakEvent`	1 par bloc non posé par un joueur
`POSER_BLOC`	matériaux	`BlockPlaceEvent`	1
`RECOLTER`	matériaux	`BlockBreakEvent`	cultures mûres (blé, carottes, pommes de terre : data 7 ; verrues : 3 ; cacao : stade 2), melons et citrouilles non posés, cannes et cactus au-dessus de leur base
`TUER_MOB`	entités	`EntityDeathEvent` (tueur joueur)	1, mobs de spawner / d'œuf ignorés
`TUER_JOUEUR`	relation : `ENEMY` , `ALLY` , `MEMBER` , `NO_FACTION`	`EntityDeathEvent` (victime joueur)	1, sauf même IP ou même victime trop récente
`INFLIGER_DEGATS`	entités (`PLAYER` ...)	`EntityDamageByEntityEvent` (coup direct ou projectile)	dégâts finaux arrondis, plafonnés à la vie restante
`PECHER`	matériaux	`PlayerFishEvent` (`CAUGHT_FISH`)	1
`CRAFTER`	matériaux	`CraftItemEvent`	quantité du résultat × nombre de crafts (shift-clic estimé d'après les ingrédients)
`FONDRE`	matériaux	`FurnaceExtractEvent`	quantité retirée
`ENCHANTER`	matériaux	`EnchantItemEvent`	1
`BRASSER`	matériaux (potions `POTION:data`)	`BrewEvent` + dernier joueur ayant ouvert l'alambic	1 par potion obtenue
`CONSOMMER`	matériaux	`PlayerItemConsumeEvent`	1
`ELEVER`	entités	`CreatureSpawnEvent` (`BREEDING`) + joueur ayant nourri un parent proche dans la minute	1
`APPRIVOISER`	entités	`EntityTameEvent`	1
`TONDRE`	entités	`PlayerShearEntityEvent`	1
`PARCOURIR`	aucune	`PlayerMoveEvent`	blocs horizontaux (hors vol, véhicule selon option, 5 blocs max par évènement)
`TEMPS_EN_LIGNE`	aucune	tâche chaque minute	1 minute par joueur non AFK
`GAGNER_EXPERIENCE`	aucune	`PlayerExpChangeEvent`	points d'expérience
`MONTER_NIVEAU`	aucune	`PlayerLevelChangeEvent`	niveaux gagnés

Type	Cibles	Source	Quantité
`LIVRER`	matériaux	clic sur la quête dans le menu	objets retirés de l'inventaire (sans nom ni lore personnalisés)
`CLAIMER`	aucune	`LandClaimEvent` de KoriaFactions	1 par chunk

Tous les écouteurs sont en priorité `MONITOR` et ignorent les évènements annulés (protection, claims refusés).

Traitement d'une action (`ProgressService.handle`)

1. Aucune quête chargée n'utilise ce type → rien (évite de travailler sur les évènements fréquents).
2. Joueur hors ligne, monde désactivé ou créatif (option) → rien. Type de livraison → rien (menu uniquement).
3. Pour chaque quête joueur non terminée du même type : filtres `mondes`, `zones` (calculée une seule fois), cibles.
4. Même chose pour les quêtes de la faction du joueur, si elle est éligible (quêtes de faction actives et `membres-minimum` atteint).
5. Avancement : `QuestProgressEvent` (annulable, quantité modifiable), ajout borné à l'objectif, contribution enregistrée (faction), barre d'action, message aux paliers de pourcentage.
6. Objectif atteint : réussite (voir ci-dessous).

Réussite

- **Quête joueur** : récompenses de la quête (ou de la difficulté), `POINTS` crédités à la faction actuelle du joueur (perdus s'il n'en a pas, avec un message), messages et son, `QuestCompleteEvent`, puis chaque palier de `joueur.<difficulté>.paliers` atteint et non reçu (`BonusEarnedEvent`).
- **Quête de faction** : `POINTS` à la faction, récompenses personnelles selon la `cible`, message aux membres connectés (`prevenir-la-faction`), annonce globale optionnelle, `QuestCompleteEvent`, puis paliers de `faction.paliers`.

Ajouter un type d'objectif

Dans le plugin : une constante et un `register(new ObjectiveType(id, cibles))` dans `ObjectiveRegistry`, puis un gestionnaire dans `ObjectiveListener` qui appelle :

```
plugin.progress().handle(ProgressContext.at(joueur, "MON_TYPE", quantite, position).material(id, data)); // ou .key("NOM")
```

Ajouter aussi la valeur dans les choix de `*.*.type` de `descriptions.json` pour l'éditeur. Depuis un autre plugin : voir l'API (section 12). `ObjectiveType.Targets` : `NONE` (tout compte), `MATERIALS` (comparaison `MaterialSet`), `NAMES` (comparaison sans casse avec `ProgressContext.key`).

9. Anti-abus

Protection	Fonctionnement
Blocs posés	chaque bloc posé (sauf plantes et pousses) est mémorisé ; cassé, il ne compte pas et est oublié. Mémoire bornée (`blocs-poses-memoire-max`, les plus anciens partent), 16 mondes au plus, sauvegardée à l'arrêt

Protection	Fonctionnement
Mobs de spawner / d'œuf	métadonnée `koriaquest-ignore` posée à l'apparition ; les mobs supplémentaires de l'amélioration SPAWNERS de KoriaFactions (`koriafactions-spawner`) sont traités pareil
Kills	ignorés si même IP (option) ou si la même victime a déjà compté depuis moins de `delai-meme-victime-minutes`
AFK	actif = mouvement de caméra ou message de chat depuis moins de `afk-minutes`
Créatif, mondes	`ignorer-creatif` , `mondes-desactives`
Livraison	objets renommés ou avec lore refusés

10. Commandes et permissions

`/quetes` (nom et alias dans `config.json` → `commande` , enregistrée dynamiquement). Sans argument : menu du joueur (ou choix de la difficulté s'il ne l'a pas fait). Chaque sous-commande lit `commandes.<id>` (`actif` , `alias` , `permission`) ; usage et description dans `messages.json` → `help.commands.<id>` . Permissions déclarées `TRUE` par défaut pour les commandes joueur, `OP` pour l'administration.

Sous-commande	Permission par défaut	Rôle
`aide`	koriaquest.command.aide	aide filtrée selon les permissions
`menu`	koriaquest.command.menu	quêtes du joueur
`faction`	koriaquest.command.faction	quêtes de la faction
`difficulte [id]`	koriaquest.command.difficulte	menu de choix, ou choix direct
`temps`	koriaquest.command.temps	temps avant les nouvelles quêtes joueur et de faction
`voir <joueur>`	koriaquest.admin.voir	quêtes d'un joueur et de sa faction
`progression <joueur|faction> <nom> <n°> <valeur>`	koriaquest.admin.progression	fixe l'avancement ; atteindre l'objectif valide la quête avec ses récompenses
`reinitialiser <joueur|faction> <nom>`	koriaquest.admin.reinitialiser	remet à zéro la période en cours d'un joueur (nouveau choix) ou d'une faction (nouvelles quêtes)
`renouveler [tout|faction|joueur]`	koriaquest.admin.renouveler	renouvellement immédiat pour tout le monde : nouveau tirage de faction et / ou nouvelles quêtes joueur
`config [fichier]`	koriaquest.admin.config	éditeur en jeu
`sauvegarder`	koriaquest.admin.sauvegarder	sauvegarde asynchrone
`recharger`	koriaquest.admin.recharger	recharge la configuration et liste les problèmes

`plugin.yml` déclare seulement les groupes `koriaquest.*` , `koriaquest.player` et `koriaquest.admin` .

11. Menus

Même base que KoriaFactions (`Menu`) : tout vient de `menus.json` , clics par type (`action` , `clic` , `clic-gauche` , `clic-droit` , `shift-clic` , `touche-q` , `ctrl-q` …), actions `COMMANDE:` , `CONSOLE:` , `MENU:` , `MESSAGE:` , `FERMER` , `ACTUALISER` , pages, plus :

Action	Effet
`CHOISIR:<difficulté>`	choix de la difficulté (confirmation par un second clic si activée)
`LIVRER`	sur une quête : livre les objets de l'inventaire (sans effet si la quête n'est pas de type `LIVRER`)
`MENU:joueur` , `MENU:difficulte` , `MENU:faction` , `MENU:config[:fichier]`	navigation

`QuestListMenu` place les quêtes aux `quetes.cases` avec le modèle `quetes.en-cours` ou `quetes.terminee` , puis les `objets` libres. Variables disponibles :

Portée	Variables
Tous les menus	`{player}` , `{joueur}` , `{faction}` , `{label}` , `{temps_joueur}` , `{temps_faction}` , `{temps_restant}` (période joueur dans les menus joueur et difficulté, semaine de faction ailleurs), `{page}` , `{pages}`
Quête	`{quete}` , `{description}` (liste), `{progression}` , `{objectif}` , `{restant}` , `{pourcentage}` , `{barre}` , `{difficulte}` , `{icone}` , `{type}` , `{points}` , `{recompenses}` (liste), `{livrer}`
Quête de faction	`{contribution}` , `{contributeurs}` (liste, format `textes.contributeur`)
Menus joueur et faction	`{difficulte}` , `{terminees}` , `{total}` , `{restantes}` , `{bonus}` (liste), `{bonus_etat}` , `{points}` (faction)
Menu difficulté	`{nombre}` , `{recompenses_<id>}` , `{bonus_<id>}` , `{nom_<id>}` pour chaque difficulté

Une variable liste produit une ligne par élément ; une ligne contenant seulement une variable vide disparaît (ex. `{livrer}` sur une quête qui n'est pas une livraison).

12. API

`KoriaQuestAPI`

Méthode	Rôle
`isAvailable()`	le plugin est-il activé
`registerObjectiveType(id, Targets)`	déclare un type utilisable dans les fichiers de quêtes (faire `/quetes recharger` si les quêtes ont été chargées avant)
`progress(ProgressContext)`	signale une action
`getDifficulty(Player)` , `getPlayerData(Player)`	données joueur de la période en cours

Méthode	Rôle
`getFactionData(String factionId)`	quêtes de la faction, ou null si elle n'est pas éligible
`getPlayerTimeLeft()`, `getFactionTimeLeft()`	millisecondes avant les nouvelles quêtes joueur / de faction

Exemple :

```
KoriaQuestAPI.registerObjectiveType("GAGNER_KOTH", ObjectiveType.Targets.NAMES);
// quand un joueur gagne un KOTH :
KoriaQuestAPI.progress(ProgressContext.of(joueur, "GAGNER_KOTH", 1).key("KOTH_CENTRE"));
```

Évènements

Évènement	Moment	Particularités
`QuestProgressEvent`	avant chaque avancement (livraison comprise)	annulable, `setAmount`, `getFactionId()` null pour une quête joueur
`QuestCompleteEvent`	après une réussite et ses récompenses	`getPoints()` crédités, `getPlayer()` null pour une validation admin
`BonusEarnedEvent`	après un palier	`getScope()`, `getTier()`, `getPoints()`
`WeeklyResetEvent`	après une nouvelle semaine de quêtes de faction	`getCycle()`, `getFactionQuests()`
`PlayerQuestsResetEvent`	après le renouvellement des quêtes joueur	`getPeriod()`, `isDaily()`

Évènements de KoriaFactions écoutés : `LandClaimEvent` (objectif `CLAIMER`), `FactionDisbandEvent` (suppression des données). KoriaFactions reçoit les points via `KoriaFactionsAPI.addPoints(faction, n, "koriaquest")`, ce qui déclenche son `FactionPointsChangeEvent`.

13. PlaceholderAPI

Identifiant `koriaquest` :

- `%koriaquest_difficulte%`, `%koriaquest_terminees%`, `%koriaquest_total%`, `%koriaquest_temps_joueur%`, `%koriaquest_temps_faction%`
- `%koriaquest_faction_terminees%`, `%koriaquest_faction_total%`
- `%koriaquest_quete_<n>_<nom|progression|objectif|pourcentage|termine>%`
- `%koriaquest_faction_quete_<n>_<nom|progression|objectif|pourcentage|termine>%`

14. Points d'attention

- Aucun test sur serveur n'a encore été fait : à valider en jeu (cycle, tirages, livraison, récompenses hors ligne).
- `KoriaQuestAPI.progress` et `ProgressService.handle` doivent être appelés sur le thread principal.

- Une récompense `ARGENT` sans économie Vault n'est pas donnée (avertissement console).
- Les objectifs de faction ne sont pas recalculés si la faction change de taille pendant la semaine.
- Les récompenses `CONTRIBUTEURS` ne vont qu'aux contributeurs encore membres au moment de la réussite.
- Conventions : clés d'options en français avec tirets, clés de messages en anglais, commentaires en français.